

PDF output writer “deep multi-angle research (Â§11.4.150)

Revision: 1 **Last modified:** 2026-06-15T00:00:00Z **Scope:** unified-translator PDF (.pdf) OUTPUT support “ pkg/ebook/pdf_writer.go + cmd/unified-translator/main.go output-format switch. **Authority:** constitution Â§11.4.150 (deep-research-first), Â§11.4.99 (latest-source), Â§11.4.6 (no-guessing), Â§11.4.3 (honest typed error on unavailable dependency), Â§11.4.1 (no broken/blank artifact).

Gap (FACT)

unified-translator supports OUTPUT formats epub / fb2 / html / htm / txt / md / docx only (cmd/unified-translator/main.go generateOutput, default case returns unsupported output format %q (supported: .epub, .fb2, .html, .txt, .md, .docx)). PDF is INPUT-only in pkg/ebook (pdf_parser.go; there is no pdf_writer.go); -o book.pdf hits the default case and is rejected. This is the last format-matrix output gap.

The load-bearing correctness constraint (FACT “ Angle 1: Unicode/Cyrillic)

This project’s PRIMARY use case is Serbian translation, including **Serbian Cyrillic** (-script cyrillic; pkg/script/). A PDF writer that silently drops Cyrillic glyphs is a Â§11.4 PASS-bluff: it produces an “openable PDF” carrying garbage/blank where the translated text should be.

PDF text rendering with the **Standard-14 base fonts** (Helvetica / Times-Roman / Courier) is restricted to **WinAnsiEncoding** (Latin-1 + a few extras). It CANNOT render Cyrillic “ confirmed against multiple authoritative sources (see Sources). To render Cyrillic (or any non-Latin-1 script) a PDF MUST embed a CID/TrueType font (Identity-H encoding + UTF-16 text + a ToUnicode CMap) whose glyf table actually contains the glyphs. Any “minimal pure-Go PDF” that uses only a Standard-14 font is therefore CORRECT only for Latin-1 content and WRONG (silent glyph drop) for the project’s own primary content.

Options considered

(a) Pure-Go minimal PDF writer with a Standard-14 font (stdlib only)

- The minimal valid PDF is small + standardized (ISO 32000): header %PDF-1.x + binary marker, 4 objects (Catalog “ Pages “ Page “ Content stream BT /F1 Tf Td (text) Tj ET), an xref table of byte offsets from file start, trailer /Root + startxref + %%EOF (verified against the PDF Association “smallest valid PDF” + Zagaeski minimal-PDF references, see Sources).
- **REJECTED for primary correctness:** Standard-14 + WinAnsiEncoding cannot render Cyrillic “ silent glyph drop on the project’s #1 content (Â§11.4.1/Â§11.4.6 violation). Latin-only output is not acceptable for a Serbian-Cyrillic translator.

(b) Pure-Go PDF writer with an EMBEDDED TrueType CID font (stdlib + a font asset)

- Would render full Unicode, but requires: (i) shipping/locating a Unicode TTF (a non-stdlib asset, or a fragile system-font-path lookup that differs per OS – §11.4.6 unknown-at-runtime),
 1. implementing TrueType subsetting / Identity-H / ToUnicode CMap / composite-font plumbing by hand. High implementation + correctness risk for a single output format; the fragile font-path lookup is exactly the §11.4.6 –œunknown without checking –œ anti-pattern.

(c) Maintained third-party Go PDF lib (go-pdf/fpdf, gopdf, maroto, gpdf)

- gopdf (jung-kurt) is ARCHIVED (2021). go-pdf/fpdf is a partly-maintained fork; gopdf (signintech) + maroto + gpdf are maintained. ALL add a new third-party dependency to go.mod, and Unicode/Cyrillic still requires supplying + embedding a TTF (AddUTF8Font). New dep + still-need-a-font-asset –†’ no advantage over (d) for this project.

(d) WeasyPrint-backed (render Book –†’ HTML –†’ PDF via the already-installed weasyprint) –œ” CHOSEN

- WeasyPrint is ALREADY an installed, used project dependency (/opt/homebrew/bin/weasyprint, v66.0; used across the doc-export pipeline for .md –†’ .pdf siblings per §11.4.65).
- Reuses the EXISTING HTML writer pattern (translate –†’ structured HTML –†’ PDF), so the writer is thin: build a UTF-8 HTML document from the in-memory Book, hand it to weasyprint.
- **Renders full Unicode incl. Cyrillic correctly** –œ” weasyprint embeds the needed system-font glyphs automatically, so the translated text is real, visible, and text-extractable (verified: Ð–Ð´ÑÐ°Ð²Ð¼ Ñ–Ð²ÐµÑ, Ðµ rendered into a valid %PDF-1.7 with %EOF).
- **Honest typed error when weasyprint is absent** (§11.4.3/§11.4.6): a WeasyPrintUnavailableError (exit-able typed error) rather than a silent broken/blank PDF –œ” the operator gets a precise, actionable message naming the missing tool, never a misnamed/empty artifact (§11.4.1).
- Trade-off (honest boundary): a runtime external-process dependency. Mitigated because weasyprint is already present + relied on by the project, and the unavailable case is an explicit typed error, not a degraded silent fallback.

Decision

(d) WeasyPrint-backed, mirroring the project’s existing HTML/doc-export pipeline. It is the only option that (1) renders the project’s own primary content (Serbian Cyrillic) CORRECTLY, (2) adds ZERO new go.mod dependency, (3) reuses the existing HTML-writer pattern, and (4) fails honestly + loudly when the tool is absent rather than shipping a broken PDF. The pure-Go options (a)/(b)/(c) either silently drop Cyrillic (a) or impose hand-rolled font-embedding + a fragile font-asset dependency (b)/(c) with no offsetting benefit.

Angle 2 – no-bigger-problem check (§11.4.150(C))

- Does PDF output regress any existing format? No – the writer is additive (case "pdf"), and the default/epub/fb2/html/txt/md/docx branches are unchanged (asserted by the format-matrix regression test).
- Does weasyprint™s HTML→PDF lose translated text? No – text is HTML-escaped (no markup injection), fonts embedded, text extractable (validity asserted: %PDF - header + page count + the translated text present in the PDF).
- Security: translated content is HTML-escaped before reaching weasyprint (no markup/JS injection); no remote resource fetching is required for plain text.

Sources verified (§11.4.99 – latest-source)

- PDF Association – “The smallest possible (valid) PDF” <https://pdfa.org/the-smallest-possible-valid-pdf/> (accessed 2026-06-15) – minimal PDF object/xref/trailer structure.
- Brendan Zagaeski – “Minimal PDF” <https://brendanzagaeski.appspot.com/0004.html> (accessed 2026-06-15) – complete minimal Hello-World PDF with content-stream text operators (BT/Tf/Td/Tj/ET) + xref offsets.
- libharu Fonts & Encodings – <https://libharu.sourceforge.net/fonts.html> (accessed 2026-06-15) – Standard-14 fonts use WinAnsiEncoding; Cyrillic/CJK require embedded Unicode TrueType (cmap/OS-2/glyf).
- pdf-lib standard-fonts / Adobe community PDF specs (accessed 2026-06-15) – Standard-14 + WinAnsiEncoding cannot render Cyrillic; CID font + Identity-H + UTF-16 required for Cyrillic.
- go PDF library comparison (gopdf archived 2021; go-pdf/fpdf fork; gopdf/maroto/gpdf maintained) – <https://www.libhunt.com/r/fpdf>, <https://gpdf.dev/> (accessed 2026-06-15) – third-party libs add a dep and still need an embedded TTF for Cyrillic.

Deep-research 2026-06-15: <https://pdfa.org/the-smallest-possible-valid-pdf/> ; <https://brendanzagaeski.appspot.com/0004.html> ; <https://libharu.sourceforge.net/fonts.html> ; <https://www.libhunt.com/r/fpdf>